Verilog Interview Questions

1. Write a Verilog module for a 2-input AND gate.
2. Implement a 2:1 multiplexer using `assign` and `case`.
3. Design a 4-bit adder using structural modeling.
4. Implement a D Flip-Flop with synchronous reset.
5. Write a testbench for a simple 2-input OR gate.
6. Design a 3-bit counter (up/down).
7. Implement a simple FSM for a traffic light controller.
8. Design an 8-bit shift register (serial-in, serial-out).
9. Write a sequence detector for the pattern "101".
10. Implement an N-bit parameterized full adder.
11. Implement a 4x1 multiplexer using `case` statement.
12. Design a 4-bit carry look-ahead adder.
13. Write a parameterized N-bit subtractor.
14. Implement a simple ALU (ADD, SUB, AND, OR, XOR, Shift).
15. Design a signed multiplier using Booth's algorithm.
16. Implement an 8-bit ripple carry adder.
17. Design a binary-to-BCD converter.
18. Design a mod-10 counter.
19. Implement a clock divider by 3.
20. Implement an asynchronous FIFO.
21. Write a Verilog module for a simple RISC CPU.
22. Design an FSM-based ATM system.
23. Implement a multi-cycle multiplier.
24. Implement a basic SDRAM controller.
25. Design a DDR3 controller with read/write functionality.
26. Implement a parameterized LFSR for random number generation.
27. Write a cache memory controller with direct mapping.
28. Design a simple pipelined processor.

29. Implement a fully parameterized AXI interconnect.
30. Design a multi-core RISC-V processor.
31. Implement a non-blocking cache for a multi-core processor.
32. Write a protocol checker for a PCI Express transaction.
33. Implement a DDR4 controller with timing constraints.
34. Design a multi-level page table for a virtual memory system.
35. Implement a full AXI4 Stream Interface with arbitration.
36. Write a scheduler for a high-speed networking processor.
37. Implement an ECC (Error Correction Code) block with Hamming Code.
38. Design a real-time video processing pipeline in Verilog.
39. Write a Verilog module to generate the sequence "10011001" repeatedly.
40. Implement a Verilog module that generates an alternating sequence of 1s and 0s.
41. Design a Verilog module to output only odd-positioned bits from an input stream.
42. Write a Verilog code to generate a repeating binary sequence "10101".
43. Implement a Verilog sequence detector that recognizes "1101" in a serial data stream.
44. Write a Verilog module that generates a Fibonacci sequence in binary.
45. Implement a Verilog module that outputs a Gray code sequence.
46. Design a Verilog module to generate a pseudo-random bit sequence using LFSR.
47. Write a Verilog code to generate a binary counter that only counts prime numbers.
48. Implement a Verilog pattern generator that alternates between "1010" and "0101" every clock cycle.
49. Code a 2:1 multiplexer using both if-else and case.
50. Write code for a D flip-flop with asynchronous reset.
51. Implement an AND gate using assign and always block.
52. Create a 4-bit binary counter with synchronous reset.
53. Write a 4-to-1 multiplexer.
54. Write a 3-bit up counter with enable and reset.
55. Design a priority encoder.
56. Write a simple shift register.
57. Code a 1-bit comparator.

58. Create a half-adder and full-adder.
59. Write a testbench for a 4-bit counter.
60. Implement a Moore FSM for even-odd detector.
61. Write Verilog code for an 8-bit barrel shifter.
62. Design a universal shift register.
63. Implement a simple ALU (AND, OR, ADD, SUB).
64. Design a binary to Gray code converter.
65. Write code to debounce a push-button input.
66. Create a 2-digit BCD counter.
67. Write a synchronous FIFO.
68. Write a dual-edge triggered flip-flop.
69. Write a Mealy machine for sequence detector (e.g., 101).
70. Implement traffic light controller using FSM.
71. Write an FSM to detect three consecutive 1's in a stream.
72. Design FSM for elevator control.
73. Create FSM for vending machine.
74. Create a ROM using case statement.
75. Write a dual-port RAM.
76. Design an asynchronous FIFO.
77. Implement an LRU cache block (simplified).
78. Code for line buffer for image processing.
79. Code for a frequency divider by 2, 4, 10.
80. Design a pulse width detector.
81. Implement one-pulse generator from a button input.
82. Create a watchdog timer.
83. Create a digital clock in Verilog.
84. Implement clock gating logic.
85. Write a code for handshake protocol.
86. Design a parameterized counter.
87. Implement edge detector.
88. Write a Gray code counter.

89. Create a UART transmitter/receiver (basic).
90. Design an SPI master.
91. Implement I2C protocol FSM.
92. Code a CRC generator.
93. Create a simple AXI-lite interface block.
94. Write a testbench for FSM.
95. Design a register file with 8 registers.
96. Implement pipelined multiplier.
97. Create priority-based arbiter.
98. Write code for pulse synchronizer between clock domains.
99. Code a ring counter.
100. Design a Johnson counter.
101. Write code for binary to BCD converter.
102. Write a GCD calculator.
103. Implement basic MAC unit.
104. Design an FSM for password checker.
105. Create a binary search FSM.
106. Implement a pipeline stall controller.
107. Write a glitch-free multiplexer.
108. Write a code to implement debounce logic.
109. Create an AXI transaction state machine.
110. Design an N-bit carry-lookahead adder.
111. Build a parameterized ALU with flags.
112. Create FSM with multiple outputs and internal states.
113. Implement time-multiplexed display driver.
114. Write synthesizable code for radix-4 booth multiplier.
115. Implement basic instruction decoder for RISC-V.
116. Create an FSM to handle serial data framing.
117. Implement a bus arbiter for 4 masters.
118. Design pipeline bypass logic.
119. Implement round-robin arbiter.

120. Create a memory-mapped register interface.
121. Design a delay line using shift registers.
122. Create an FSM with parameterized state encoding.
123. Implement a reconfigurable counter.
124. Write Verilog code for an N-bit adder using generate statement.
125. Implement an N-bit subtractor with overflow detection.
126. Write a floating-point encoder (IEEE 754 format, basic).
127. Write code for a crossbar switch (4x4).
128. Implement a bitonic sorter (4 elements).
129. Code a simple cache controller FSM.
130. Design a basic DMA controller.
131. Create an AXI-lite Write FSM.
132. Implement a 2-flop synchronizer.
133. Write a multi-bit synchronizer.
134. Design CDC safe pulse synchronizer.
135. Create reset synchronizer logic.
136. Code handshake synchronizer between two clock domains.
137. Design a power-on reset generator.
138. Implement low-power clock gating cell.
139. Write power sequencing logic for subsystems.
140. Code programmable glitch-free mux for clock sources.
141. Create brown-out detector logic in Verilog.
142. Add assertions for a FIFO (full, empty, overflow).
143. Create a timeout check for FSM lock-up.
144. Write an assertion to check illegal state in FSM.
145. Implement a simple event counter for debug.
146. Add a logging module that logs transitions to file.
147. Code an interrupt controller.
148. Create an FSM-based traffic arbiter for buses.
149. FSM to handle input handshake protocol with ACK/NACK.
150. FSM to parse a serial data stream with headers and footers.

151. FSM with timeouts (using counters).
152. FSM for automatic door controller.
153. FSM to detect pause-resume-reset command sequence.
154. Implement Binary to Gray and Gray to Binary converter.
155. Design a binary to Excess-3 converter.
156. Create a BCD to Binary converter.
157. Implement 7-segment display driver for BCD input.
158. Code for positive edge detector.
159. Code for negative edge detector.
160. Write logic to detect rising and falling edge simultaneously.
161. Create glitch-free edge detection logic.
162. Write a pulse stretcher module.
163. Create a parallel adder using a loop.
164. Design a dynamic priority encoder using a loop.
165. Implement a priority decoder using a loop.
166. Design a shift register with variable length using parameterization.
167. Implement a basic SPI Slave module.
168. Write a Verilog UART Baud Rate Generator.
169. Design a multiplier using shift-add method.
170. Write a carry-save adder.
171. Code a leading zero counter.
172. Design a parity generator and checker.
173. Create a priority encoder with mask input.
174. Design a programmable counter (with variable step size).
175. Write code for signed multiplication using Booth's algorithm.
176. Implement a Wallace Tree multiplier (conceptual level).
177. Implement a 3-stage pipelined multiplier.
178. Design a pipeline register with enable and reset.
179. Create a generic pipeline wrapper module.
180. Write a stall control logic for a pipeline.
181. Implement a hazard detection unit.

182. Code a basic I2C Master FSM with ACK/NACK logic.
183. Implement a Wishbone-compliant register interface.
184. Create a single-port synchronous RAM using array.
185. Create dual-port synchronous RAM.
186. Implement memory with byte enable support.
187. Write a content-addressable memory (CAM) model.
188. Implement a block RAM with initialization from a file.
189. Write a testbench to verify a barrel shifter.
190. Create stimulus with random data using $random.
191. Write a self-checking testbench.
192. Generate waveform output with $dumpvars.
193. Write a scoreboard module in testbench.
194. Implement fixed-priority arbiter for N requesters.
195. Implement round-robin arbiter using FSM.
196. Design a weighted round-robin scheduler.
197. Write a simple APB slave interface.